Prove Less, Repair Better — Mid-Submission Progress Report

Automation-Friendly Program Repair for Small Models in Lean 4

Agents of Ultron: Inesh Dheer (2023111010) Poojitha J (2024101088)
Gracy Garg (2023101118)

Agentic Software Engineering (CS6.403), IIIT Hyderabad Repository: `https://github.com/amoiba42/Leanstein`

6 November 2026

Summary

We proposed testing whether small, locally runnable models produce more *verified* program repairs in Lean 4 when they repair only the code and leave the proof to a fixed, pre-registered portfolio of Lean automation (condition C), rather than writing proofs themselves (B). By this milestone the project has gone well past a prototype. The harness is complete and tested; we built a buggy-VERINA dataset with hidden tests; we ran a pilot and a power analysis; we froze the configuration; and we ran the **full pre-registered study (12,249 runs)** and its analysis, all on one 48 GB MIG slice of an NVIDIA RTX PRO 6000 Blackwell. The main result supports the hypothesis: C beats B for both primary models ($p_{\text{Holm}} < 0.001$). Against a proof-writing agent that gets the same automation first (BP), C is level or slightly better, and cheaper in tokens and (estimated) energy. The remaining work is human confirmation of the audit, code-shape examples and polishing the paper.

1 What has been built

Harness (`prl/`, 5.4k lines of Python; 96 unit and 24 Lean integration tests, all passing).

- *Tasks* (`prl/tasks`): loads VERINA at a pinned commit, splits each task into locked regions (specification, theorem statement, tests) and editable ones (code, helpers, proof), and checks eligibility.
- *Bugs* (`prl/natural`, `prl/mutate`): *natural* bugs are zero-shot implementations by Qwen3.5-0.8B/9B that compile and fail a test; *mutants* come from 8 operators (arithmetic, relational, boolean, constant, condition negation, branch swap, off-by-one, argument swap).
- *Hidden tests* (`prl/testgen`): 30 generated inputs per family whose precondition is *proved in Lean*, with expected outputs from the reference. They are never shown to a model.
- *Verification* (`prl/lean`): a persistent Lean 4.24 + Mathlib server per worker; static integrity checks (no `sorry`, axioms, `native_decide`, locked-name redefinitions, or proof text in code-only conditions); an axiom check; and a frozen two-tier tactic portfolio (Tier L: `decide`, `simp`, `simp_all`, `omega`, `split`; Tier H: `grind`, `aesop`, induction templates).
- *Conditions and driver* (`prl/conditions`, `prl/driver`): conditions A, B, BP, C, C-plain, S, S-plain, Bhammer and T behind one interface, with matched budgets (8 generations, 48k tokens, 600 s of inference). Infrastructure failures are retried without charging the model, and every attempt is logged with its tokens, model time and Lean time.
- *Analysis* (`prl/analysis`): family-level paired sign-flip tests with Holm correction, cluster bootstrap, power simulation, shaping-vs-reverting classification, code-shape features, B→C cross-evaluation, and the audit sampler.
- *Pre-registration* (`prereg/`): study parameters, portfolio, prompts and power note, hashed by `prl freeze`. The driver refuses confirmatory runs on any mismatch, and changes go to `prereg/deviations.md`.

Figure 1 shows the architecture. Design documents (requirements, HLD, LLD, architecture decisions, implementation plan) are in `docs/`, and the run record is in `scripts/irel/README.md`.

Dataset. 189 VERINA tasks were screened and 184 are eligible (104 basic, 80 advanced). They are split by family into pilot (20), main (114) and held-out (50). Natural bugs cover 33 `main` families. Because that is below the pre-registered 60, the fallback pool (natural ∪ mutant, 106 families) applies. The portfolio alone proves 22.3% of eligible references (basic 35.6%, advanced 5.0%); the screening gate needed ≥20%.

2 Demonstration on a representative task

`verina_basic_1` asks whether two integers have opposite signs. Its postcondition is $((a < 0 \wedge b > 0) \vee (a > 0 \wedge b < 0)) \leftrightarrow \textit{result}$. The natural bug (from Qwen3.5-9B) returns `true` for $a = 0, b = -10$:

```
if a < 0 then if b > 0 then true else false
else if b < 0 then true else false
```

B (Qwen3.5-4B, seed 11) fixes the code on its second attempt (`(a < 0 ∧ b > 0) ∨ (a > 0 ∧ b < 0)`). It then spends all remaining attempts on proofs that begin with `intro`, which Lean rejects every time ("no additional binders"), and ends unresolved with correct code. **C** submits the same code on its *first* attempt. It passes all visible and hidden tests, the portfolio's first Tier-L entry (`simp`) proves it, and the run ends as a verified repair. This pattern is common: when B's code compiled but its proof failed,

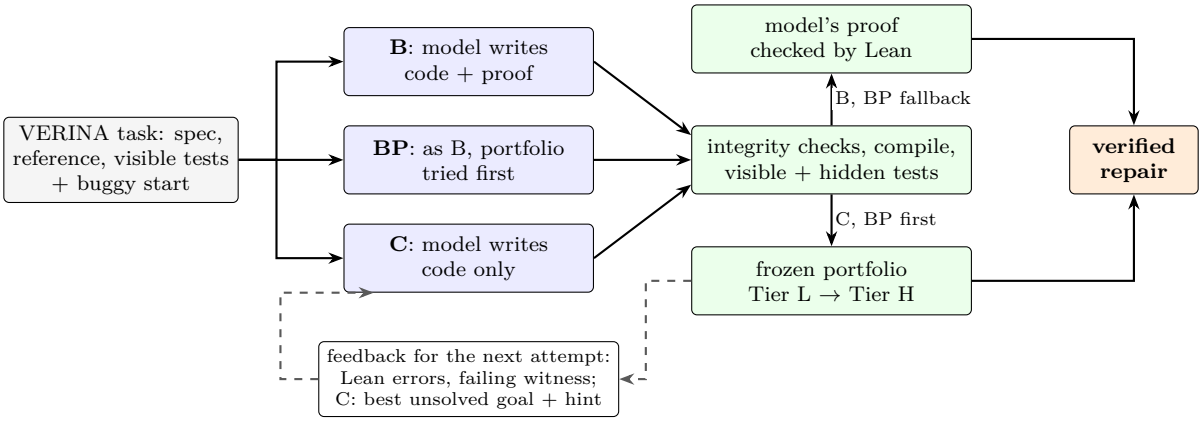


Figure 1: Method. Every condition gets the same buggy start, the same visible information and the same budget (8 generations, 48k tokens, 600 s of inference). A candidate must pass integrity checks, compile and pass every visible and hidden test before any proof is tried; hidden tests are never shown to the model. In B the model's own proof must check; in C the model writes no proof and a frozen two-tier portfolio of Lean automation must prove the unchanged specification; BP tries the portfolio first and falls back to the model's proof. A failed attempt returns feedback for the next one (dashed), in every condition except the single-shot baseline A.

C's pipeline would have verified 31% of those attempts (4B).

3 Evaluation results to date

Table 1: RQ1 (confirmatory), `main` split, 106 families, family-level verified-repair rate. One-sided sign-flip test, Holm over the four tests, 95% family-cluster bootstrap CI.

Model	Test	C	Other	Diff. [95% CI]	p_{Holm}
Qwen3.5-2B	C vs B	27.7	2.2	+25.5 [18.2, 33.0]	<0.001
Qwen3.5-2B	C vs BP	27.7	22.6	+5.0 [1.6, 8.8]	0.012
Qwen3.5-4B	C vs B	33.0	17.6	+15.4 [9.4, 21.7]	<0.001
Qwen3.5-4B	C vs BP	33.0	32.7	+0.3 [−2.8, 3.5]	0.50

The study ran 12,249 runs from 5 Nov 21:06 to 6 Nov 11:49: `main` 6,930 (Qwen3.5-2B/4B × A, B, BP, C, C-plain × 3 seeds), held-out 1,710, study S 1,068, Gemma 693, Goedel 462 and Bhammer 1,386. There was 1 infrastructure failure, completed on resume; no run hit the time cap and no stop rule fired.

- **RQ1** (table 1 and fig. 2): C beats B clearly for both models. C vs BP is +5.0 pp for 2B and a tie for 4B, as the pre-freeze power analysis predicted (power 0.71 and 0.08). The **held-out split** replicates C over B (+18.8 and +19.6 pp, $p_{\text{Holm}} = 0.001$), and C vs BP ties there for both models.
- **RQ2, mechanism**: many C successes are *shaping*, meaning the reference itself is not provable, so the repair must change the code's shape (81 of 191 for 4B; 56 are reverts). Re-shaping *correct* code (study S) is rare and hint-independent (4B: 12.7% in both S and S-plain).
- **RQ3, feedback**: the structured hint does not beat a plain retry (C vs C-plain: −1.3 and +0.3 pp, n.s.).
- **RQ4, code shapes**: recursion defeats the automation. The portfolio proves 0 of 101 recursive references against 49% of the rest. `match`, `a[i]!` indexing and `Nat`/`Int` casts also hurt.
- **RQ5, cost and scale**: for 4B, C uses 37k tokens per verified repair against 65k for BP and 165k for B, and 68 s of inference per run against 162 s and 150 s. Estimated energy per verified repair (calibrated on our slice, D-4) is 2.6 kJ for C against 5.9 kJ for BP and 13.4 kJ for B (2B: 2.0, 3.6 and 56 kJ). The whole study used about 3.1 kWh (2.3 kg CO_2). Gemma-4-E4B shows the same pattern (C 30.2%, B 1.9%, BP 27.4%). The 8B proof specialist Goedel-Code-Prover (A and B only) verifies 16.5–20.8% of runs, below Qwen3.5-4B in C (27.6%).
- **Is B a strawman?** B stays well below C even when single-call automation proofs are not counted (B_noauto) and when it is told the tactics (Bhammer, 24.2% vs C 33.0% for 4B). The audit found no specification gaming, and no run was verified but behaviourally wrong.

By bug type (table 3): local operator and constant bugs benefit most, as H2 predicted. Natural bugs are harder for every condition, but only C and BP repair them at all.

4 Known failures and limitations

- **Energy is an estimate (D-1, D-4).** The in-run meter read the NVML counter of physical GPU 0 instead of our slice, because NVML ignores the CUDA device mask, and its CPU figure mixed two concurrent jobs. Its numbers were withdrawn. After the study we calibrated joules per prompt and completion token for each model on our slice, using study prompts and settings, three

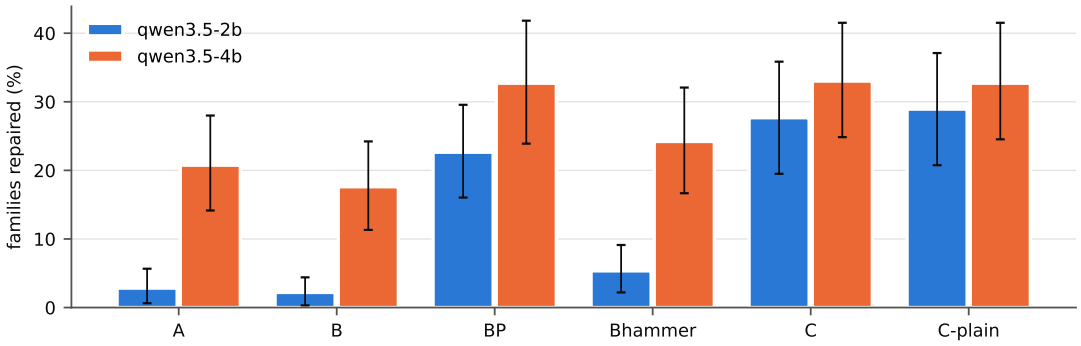


Figure 2: Family-level verified-repair rate per condition (`main`, primary bug pool), with 95% bootstrap intervals. Bhammer is exploratory.

Table 2: Original proposal vs. the study as built and run. Reasons are in `docs/final_proposal.md` and `prereg/deviations.md`.

Aspect	Original proposal	Now
Models	Qwen2.5-Coder 1.5B and 3B, 4-bit, 8 GB laptop GPU	Qwen3.5-2B and 4B (primary), Gemma-4-E4B (second family), Goedel-Code-Prover-8B (proof specialist); all 4-bit
Hardware	one laptop	one 48 GB MIG slice of an RTX PRO 6000 Blackwell, used for every step from dataset to analysis; an earlier freeze prepared on a laptop was superseded before any result was read
Conditions	A, B, C, optional D (progressive hints)	added BP (portfolio first, then the model's proof), C-plain (replaces D as the feedback test), S/S-plain (re-shaping correct code), T (screening), Bhammer (exploratory)
Bugs	injected defects only	natural bugs from other models as the primary pool, mutants as fallback and stratum
Hidden tests	independent behavioural tests	generated inputs with Lean-*proved* preconditions
Sampling	$k = 4$ candidates per round	2 per round in C, so total budgets match B
Scale	60–100 families	114 `main` + 50 held-out families, 3 seeds; held-out analysed once, separately
Rigour	pre-registered portfolio	full pre-registration with a hash freeze, pilot power simulation, Holm correction, stop rules and a public deviation log
Added analyses	—	shaping vs reverting, B_noauto, B→C cross-evaluation, code-shape catalogue, integrity audit
Time cap	—	600 s of inference per run; a calibration showed a 240 s cap was not condition-neutral

Table 3: Per-run verified-repair rate (%) by bug type, `main`, Qwen3.5-2B and 4B pooled (the original H2). n = buggy instances.

Bug type	n	A	B	BP	C
arithmetic operator	44	14.8	11.4	36.4	40.2
constant	39	17.5	15.8	26.5	35.5
boolean operator	7	4.8	0.0	9.5	26.2
relational flip	25	7.3	4.7	16.7	20.0
negated condition	57	3.5	2.6	19.0	17.3
swapped branches	16	1.0	0.0	14.6	16.7
natural (model-written)	34	2.9	0.0	9.8	15.7
off-by-one / arg. swap	9	1.9	5.6	11.1	7.4

cycles, and marginal board power over idle. We then charged every logged model call accordingly. Two jobs ran side by side in the study, which may raise absolute figures by up to ≈1.6×, but equally for every condition.

- **Human audit pending (D-3).** An automated pre-screen of all 333 sampled successes found 0 spec-gaming and 27 correct but unnatural runs; the human audit is still pending. The spec-copying flag fires on correct re-statements of executable postconditions, so its threshold is too low for VERINA.
- **Statistical power.** C vs BP is underpowered by VERINA's fixed size (114 `main` families), so the 4B tie is reported as a tie, not as evidence of no difference.
- **Coverage.** Natural bugs reach only 33 families (hence the fallback pool), and no condition repairs any of the 10 held-out natural-bug families. The portfolio proves 5% of advanced references and none that use recursion.
- **Integrity rejections** are frequent in proof-writing conditions (5.7k rejected attempts in B against 1.7k in C). Most are malformed outputs (lemmas in the helper region, duplicate definitions), not cheating.

- **Two analysis bugs** found after the runs were fixed and logged (D-2): the audit sampler crashed on a pandas name clash and dropped a column. Neither affects a verdict.
- **Scope.** The models are 4-bit only, the functions are small, and everything ran on one GPU model.

5 Changes from the original proposal

Table 2 summarises the changes. They strengthen the test of the hypothesis: BP isolates "do not write proofs" from "automation exists", and natural bugs, held-out replication and the freeze guard against over-fitting the method to the benchmark.

6 Milestones and remaining work

Achieved (dates in 2026): repository and v2 design (2 Nov); harness and tests (2–3 Nov); review revisions (cross-evaluation, power simulation, Bhammer; 3 Nov); the seven team decisions as ADRs and the first freeze (4–5 Nov); a from-scratch rerun on the GPU server, with dataset, probes, a 492-run pilot, power note and second freeze (5 Nov, 21:06); the full study, analysis and paper draft (6 Nov); the post-hoc energy calibration on our slice (6 Nov, 14:45–15:30). `docs/implementation_plan.md` tracks every task.

Repository and commits. The repository (`https://github.com/amoiba42/Leanstein`) has team commits on `main` since 2 Nov, plus review branches (`alternate`, `working`) whose changes were merged as revision 3. In order: the initial repository and v2 design, docs and harness; the revision-3 additions; the ADRs for the seven team decisions; the first freeze and run handover; and, with this submission, the IREL dataset, run logs, results, paper and this report. Generated artefacts (`data/`, `runs/` without raw model outputs, `results/`) are committed as research artefacts so that every number can be traced back to a run.

Remaining before the final submission:

1. Human confirmation of the 333-run audit.
2. Annotated before/after examples for the code-shape catalogue (RQ4).
3. Paper polish and a venue choice (a workshop such as LLM4Code or FormaliSE). An optional CLEVER replication if time allows.

The paper draft with every result filled in is `paper/main.pdf`. All results are in `results/`, and all run logs except raw model outputs are in `runs/`.

References

[1] Z. Ye et al. VERINA: Benchmarking verifiable code generation. ICLR, 2026.